



DarkNight Home Theater

DISCORD-NATIVE SOCIAL MOVIE THEATER

Watch movies together inside Discord — a real theater with seats, avatars, popcorn, synced playback and host controls. Streamed from your own device. No Cloudflare, no cloud storage, no permanent library.

Discord Activity

discord.js v14

Local range streaming

Temporary host sessions

Animated Canvas pre-show

Replit / Railway ready

Project rundown & deployment guide · repo [kaosregulator/Dark-Night-Theater](#) ·
branch [main](#)

1 · What it is

A **Discord-native movie theater**. Friends join a voice channel, open the Theater **Activity**, take a seat, grab popcorn 🍿, and watch a movie **together, in sync**. One host controls play/pause/seek. The movie streams **straight from the host's device** — nothing is uploaded to a cloud video service.

The one platform reality it's built around

Discord can't play a video inline inside a text message, and bots can't stream video into a channel. The **only** supported way to get real, synced video inside Discord is a **Discord Activity** in a voice channel (the same tech as "Watch Together"). So the design uses **two surfaces that share one state**:



Text channel

`/watch`, `/theater` — browse, launch, and **live control buttons** (▶️ ⏸️ ⏮️ ⏭️ 📺 🔒). Only the host's presses count.



Voice-channel Activity

The actual theater: the video, seats with Discord avatars, popcorn, and synced playback.

Both are keyed by the **voice channel id**, so pressing ▶️ in the channel moves the video for everyone in the theater, and vice-versa.

2 · Everything it does

- `/watch` — browse the library (title, duration, category).
- **Host a Movie from your device** — one tap inside `/watch`.
- **Animated pre-show** — Box Office → Ticket → Popcorn → Seat.
- **Watch Party (clan)** — synced playback for the whole channel.
- **Private viewing** — solo, with resume + watch history.
- **One host controls** play/pause/seek, with 🔒 lock.
- **Live control panel** in the text channel, auto-refreshing.
- **The theater** — screen, curtains, seats+avatars, 🍿🗑️.
- **1–3hr+ videos** via HTTP range streaming + seeking.
- **Per-server settings** (`/theater-settings`).

3 · The in-Discord pre-show (animated)

When a host starts a movie, a **public, self-deleting** message plays a sequence of **animated Canvas GIFs** in the channel — each showing the user's Discord avatar — then vanishes as they drop into the movie. Only the person who started it can press the buttons; everyone else is nudged to run `/watch` themselves.

(PDF pages are static — the frames below are stills; the real thing animates. The four `gif-*.gif` files delivered with this document are the live versions.)



1 · Box Office — curtains part to reveal the movie



2 · Ticket prints with your avatar + seat



3 · Concession stand — popcorn pops (random snack)



4 · Seated — you drop into a seat, lights dim → movie

4 · How a watch party works

- 1 Host runs `/watch` in a voice channel and taps 📺 **Host a Movie (from my device)**.
- 2 A one-tap, pre-authorised page opens (no key — it already knows the server + voice channel). Host picks a video file.
- 3 The **party starts immediately** and the movie **streams to viewers while it's still uploading** from the host's browser.
- 4 Everyone opens the Theater in the voice channel. The host drives ▶/⏸/⏮/⏭; viewers stay in sync.
- 5 When the party ends (or after idle/TTL), the temporary server copy is **automatically scrubbed**. The original never left the host's device.

5 · Hybrid temporary-host model & reliability

- **Temporary per-party file** — no permanent library; each party gets an ephemeral file, auto-deleted on end / ~30 min idle / TTL (6h default) / on restart.
- **Stream-while-uploading** — HTTP range (206). Seek-behind is instant; seek-ahead of the upload waits briefly for bytes.
- **Resumable upload** — a network hiccup auto-continues from the server's offset without restarting the movie or the pre-show.
- **Host status notices** — viewers see "⚠ Host connection lost — waiting for the host..." or "⌚ Buffering..."; position & buffer are preserved and it resumes automatically.

Two honest limits (documented): the host's browser tab must stay open (the movie streams from their device), and progressive "watch-while-uploading" is smoothest with **WebM** or **faststart MP4** (moov atom at the front). Any MP4 still plays & seeks once enough is buffered.

6 · Deploy on Replit (or Railway)

The repo is deployment-ready: `.replit` + `replit.nix` tell Replit to `npm install`, `npm run build`, then `npm start` — so on import it installs and runs with no manual setup.

1 Import the repo `kaosregulator/Dark-Night-Theater` into Replit (branch `main`).

2 Add these **Secrets** (this is the full required set):

Secret	Where to get it
<code>DISCORD_BOT_TOKEN</code>	Discord Dev Portal → your app → Bot → Reset Token
<code>DISCORD_CLIENT_ID</code>	Dev Portal → General → Application ID
<code>DISCORD_CLIENT_SECRET</code>	Dev Portal → OAuth2 → Client Secret (needed for the Activity)
<code>PUBLIC_BASE_URL</code>	Your Replit URL, e.g. <code>https://dark-night-theater.you.repl.co</code>
<code>SESSION_SECRET</code>	Any long random string (signs playback tokens). Recommended.
<code>HOST_ADMIN_KEY</code>	Optional — protects the admin uploader; falls back to <code>SESSION_SECRET</code> .

3 In the Discord Dev Portal → **Activities** → **URL Mappings**, add one row: `/` → your host (no `https://`). Enable Activities.

4 Run once in the Replit shell: `npm run register` (registers the slash commands).

5 Invite the bot with scopes `bot` + `applications.commands` and permission **Create Instant Invite** (that's how it launches the Activity), plus Send Messages / Embed Links / Attach Files.

Reality check on "just token + app id": you also need the **Client Secret** (for the in-Discord Activity login) and the **public URL** (Replit gives you this). Those four are the real minimum. Everything else has safe defaults.

7 · Commands

Command	Who	What
<code>/watch [search] [category]</code>	Everyone	Browse + Host a Movie + start a party / private viewing
<code>/theater</code>	Everyone	Open the Theater in your voice channel + post live controls
<code>/library sync status</code>	Manage Server	Re-scan the media folder; show where to add movies
<code>/theater-settings</code>	Manage Server	Per-server config (toggles, host/manager roles, max viewers...)

8 · Repo map

Path	Role
<code>src/index.js</code>	Boots the web server + Discord bot
<code>src/bot/</code>	Commands, handlers (watch, theater, preshow, settings), Canvas GIFs
<code>src/media/store.js · temp.js · token.js</code>	Local library, temporary per-party sessions, signed tokens
<code>src/web/routes/</code>	<code>media</code> (range) · <code>tmedia</code> (progressive) · <code>host</code> (uploader) · <code>api</code>
<code>src/services/sessions.js</code>	Watch-party rooms: playback anchor + presence + sync
<code>client/</code>	The Discord Activity (Vite): theater UI, hls/file player, WS sync
<code>assets/fonts/</code>	Bundled DejaVu font so Canvas text renders on any host

9 · Content & liability

The bot is a **player only** — it owns and stores no third-party content. Every movie is provided by the operator from their own device/files, temporarily. Whoever supplies content warrants it is purchased/licensed and is solely liable. Pirated/illegal/unlicensed content is not supported. Provided "AS IS", no warranty. Full terms: `DISCLAIMER.md`.



Everything above is on `main`, verified building and booting, ready to import and run.